



INDIA.RUNS

Build what next India runs on

Team Name :

Team Leader Name : Anirudh Lohiya

Problem Statement : Build an offline, explainable, and CPU-efficient AI engine to rank candidate profiles against a 'Senior AI Engineer — Founding Team' role, filtering out data anomalies and keyword stuffers.

Solution Overview

- **Proposed Solution (Hybrid Multi-Stage Engine)**

Stage 1 (Local Semantic Match): Calculates semantic cosine similarity against the JD using localized TF-IDF (unigrams + bigrams) and Latent Semantic Analysis (LSA) projected to a 64D space, requiring zero remote APIs or large downloads.

Stage 2 (Heuristic Rule Scorer): Evaluates must-have competencies (vector search, ranking evaluation), nice-to-haves, preferred locations (Noida/Pune), and YOE bands (5–9 yrs) while applying soft penalties to title chasers or non-coding leads.

Stage 3 (Behavioral & Integrity Gates): Applies an availability multiplier (active recency, response rate, notice period) and intercepts inconsistent histories (overlapping employment, mismatched durations) via a Honeypot checker.

Key Differentiators from Traditional Systems

Defeats Keyword Stuffing: Analyzes complete career history and role tenures instead of simple phrase tallies, correctly downranking keyword-heavy but unqualified profiles (e.g. HR managers listing AI tools).

100% Deterministic & Safe: avoids hallucination risks, API latency, and data leakage by operating offline and locally without generative LLM. Completely s.

JD Understanding & Candidate Evaluation

- **Key Requirements Extracted from the JD**

Production Embeddings & Retrieval: Sentence-transformers, OpenAI embeddings, BGE, E5.

Vector Search Infrastructure: Pinecone, Weaviate, Qdrant, Milvus, OpenSearch, Elasticsearch, FAISS.

Ranking Evaluation Frameworks: NDCG, MRR, MAP, A/B testing, offline-to-online correlation.

Ideal Profile Characteristics: 5–9 years of experience (ideal) and based in India (Pune/Noida preferred).

Evaluating Fit Beyond Keyword Matches

Disqualifier Analysis: We model and filter out consulting-only, pure-research, and CV/Speech-only backgrounds without NLP exposure, as well as recent LangChain-hype profiles lacking pre-LLM ML depth.

Stability & Role Checking: Detects title-hopping (multiple short-tenure senior stints) and non-coding leads (roles with senior titles that lack hands-on evidence).

Ranking Methodology

- **Stage-by-Stage Ranking Framework**

- 1. Semantic Embedding Space (45% weight):** Fits TF-IDF Vectorizer (ngram_range=(1,2)) and TruncatedSVD (64 components) on candidate texts and JD. Computes cosine similarity to establish a normalized semantic overlap score.
- 2. Heuristic Scoring (55% weight):** Awards points for production ML/retrieval keywords (max 40 pts), evaluation metrics (max 15 pts), Python (5 pts), nice-to-haves (10 pts), YOE compliance (15 pts), and location (10 pts). Applies penalties for title-chasers and non-coding leads (-8 pts each).
- 3. Hybrid Combination:** Aggregates scores into ``Base Score = 0.55 * Rule_Score + 0.45 * Semantic_Score``. Hard-disqualified candidates are scaled by 0.15 to push them low while maintaining visibility.
- 4. Behavioral Scaling:** Adjusts the base score with the ``availability_multiplier`` derived from inactivity duration, open-to-work flag, response rate, and notice period.
- 5. Honeypot Penalty & Sorting:** Scales anomalous profiles by a 0.05x multiplier to push them to the absolute bottom of the pool. Sorts descending by score, using candidate ID ascending as a deterministic tie-breaker.

Explainability & Data Validation

- **Explainable Ranking Decisions**

Deterministic Explanations: Generates structured, clear reasons detailing years of experience, current titles, and exact keyword matches (e.g. `Senior AI Engineer with 5.9 yrs; 13 production ML/retrieval keywords; response rate 0.80`).

Eliminating Hallucinations: Operating locally without a generative LLM means decisions are completely traceable, stable, and audit-friendly. The logic remains transparent during inspections.

Data Integrity & Honeypot System

Chronology Checks: Flags profiles containing overlapping employment intervals (>45 days across different companies) or inverted education dates.

Implausible Durations: Detects candidates claiming multiple concurrent current roles, total tenure exceeding stated YOE, or individual skill durations exceeding total career length.

Soft-Gating Architecture: Rather than deleting suspicious profiles (which creates false negatives), we scale their final scores by 0.05 to rank them at the bottom. This preserves data transparency for human review.

End-to-End Workflow

- **Core Processing Workflow**

- 1. Candidate Ingestion:** Stream records from candidate JSONL file. Concatenate profile text fields (summary, headline, current title, job history titles/descriptions, skills) into a single corpus document.
- 2. Semantic Vector Space:** Initialize and fit TF-IDF vectorizer (using unigrams and bigrams, up to 8000 features). Apply Truncated SVD to project the sparse matrix into a dense 64D space, and calculate cosine similarity against the parsed JD vector.
- 3. Heuristic Scoring:** Extract candidate attributes and check against the JD rubric to calculate a rule-based fit score. Identify any hard disqualifiers (e.g. consulting-only) and apply a 0.15x scaling multiplier if present.
- 4. Behavioral & Honeypot Processing:** Calculate availability multiplier using engagement signals. Scan dates for timeline overlaps or implausible durations, applying a 0.05x honeypot penalty to anomalous records.
- 5. Ranking Blending & Output:** Compute final hybrid score: $(0.55 * \text{Rule} + 0.45 * \text{Semantic}) * \text{Availability} * \text{Honeypot}$. Sort descending, resolve ties by candidate ID, filter the top 100, and output to a validated CSV file.

System Architecture

Modular Architecture (Offline-First)

jd_config.py (Rubric Central): Defines the core matching keywords, locations (Pune/Noida), target experience limits (5–9 yrs), and disqualifier indicators used across the pipeline.

features.py (Feature Extractor): Parses dates, calculates tenure lengths, extracts locations, evaluates behavioral factors, and runs the 5 logical timeline honeypot rules.

scorer.py (Evaluation Blender): Computes TF-IDF & Truncated SVD, measures cosine similarities, evaluates the rubric checklist, and blends results into final candidate scores.

rank.py (Driver & CLI): Streams records, coordinates feature extraction, scores candidate documents, performs sorting, and writes formatted CSV outputs.

Architecture Highlights

High Scalability: Processes candidate profiles dynamically to minimize system memory footprint, avoiding large array loads.

No Network / GPU Required: Avoids deep-learning model downloads, providing fast execution on minimal CPU hardware with standard Python packages.

Results & Performance

- **Runtime Efficiency on 100K Profile Pool**

Execution Time: Completed in ~100 seconds, significantly below the 5-minute challenge limit.

Memory Usage: Peak RAM stayed at ~2.8 GB, well within the 16 GB system memory allocation.

Ranking Accuracy & Heuristic Validation

Sample Candidate Evaluation: Tested against the 50-candidate sample. The ranker successfully placed the single candidate possessing production recommendation-system experience as #1 (score: 1.0).

Filtering Keyword Stuffers: Unqualified candidates who stuffed AI terms into their profile summaries (e.g. non-technical managers) were correctly penalized and ranked near the bottom, avoiding keyword-matching pitfalls.

Honeypot Gate In Action: Profiles containing contradictory dates or education histories were successfully flagged and demoted, preserving output integrity.

Technologies Used

- Selected Tech Stack & Rationale

Python 3: Standard language for candidate parsing pipelines and machine learning development.

Scikit-Learn (sklearn): Powers `TfidfVectorizer` and `TruncatedSVD` (for Latent Semantic Analysis). Offers fast, robust, and reproducible vector models on standard CPU cores.

NumPy & SciPy: Optimized numerical operations for sparse matrix mathematics and cosine similarity calculations.

Standard Libraries (csv, json, datetime): Ensures lightweight execution without bulky external dependencies.

Alternative Tech Rejections

No Deep Learning Models: Transformers (e.g., Sentence-Transformers) require heavy download weights and slow down initialization on CPU servers.

No Generative LLMs: API calls for 100K profiles introduce severe cost, runtime latency, and non-deterministic ranking outcomes.

Submission Assets

Deliverable Package

GitHub Repository: Includes modular source files, dependencies checklist, and a Colab playground notebook.

submission.csv: The final formatted output containing the top 100 candidates ranked by the hybrid engine.

Approach PDF: A converted copy of this presentation document explaining the system design.

Validation & Schema Verification

Schema Verification: The output structure matches `[candidate\_id, rank, score, reasoning]` columns exactly.

Automated Integrity Checks: Validated via `validate\_submission.py` to confirm that the file contains exactly 100 candidate rows, unique ranks from 1 to 100, scores are non-increasing by rank, and ties are broken in alphabetical order of IDs.



INDIA.RUNS

Build what next India runs on

THANK YOU